Fishing Game

Complete chat history, code and test results

Part	Contents
A	Conversation: your 4 messages and my 4 answers, in order
B	Your uploaded original files (7 files)
C	Step-by-step work log: what I did and why, in both rounds
D	Diffs: original vs updated code, file by file
E	Final code of every file in the zip
F	Test script and test output (14 checks)
G	Screenshots: yours and the game frames I rendered
H	How to run, controls, and what to check on your machine

Date: 6 October 2026. Project: pygame fishing game lab (Tasks 1-4).

Part A - Conversation

Blue boxes are your messages, orange boxes are my answers. Text is as sent.

YOU - Message 1

Task 1: Fix the catch-detection bug
A fish is supposed to be caught only when the hook actually overlaps it. In the current build, `check_catch` (in `game/catch.py`) only compares the hook's depth to the fish's depth, within a fixed 10-pixel tolerance - it never compares horizontal position at all. That means a fish can register as caught while it's still far off to one side of the screen, just because the hook happens to be passing through its depth. Fix the check so it's based on the hook and fish actually overlapping, not just being at a similar depth.
Task 2: Implement multiple fish types
Introduce at least two fish types that differ in movement speed and point value (for example, a slow low-value fish and a fast high-value one). Give each type its own look (color and/or size) so they're visually distinguishable, and make sure the correct point value is awarded when each type is caught.
Task 3: Implement player-controlled casting
Change the hook so the player decides when it casts, instead of it looping automatically. Pressing a key should start a cast if the hook is currently idle; it should then travel down and return on its own (already handled) once it reaches maximum depth or catches a fish. A new cast should not be able to interrupt one that's already in progress.
Task 4: Implement a 30-second round timer
Add a 30-second countdown for the round. Display the remaining time on screen. Once it reaches zero, no further catches should be possible, the round should end, and the final score should be shown clearly. Provide a way to start a new round with the score and timer both reset. at last give updated codes zip file

CLAUDE - Answer 1

I've done all four tasks and packaged the project as fishing_game.zip. I ran it headless to test the key pieces: a far-away fish isn't caught, an overlapping fish is, a second cast during a cast is ignored, the round ends at the timer, and R resets everything. I haven't played it in a window. To run it, unzip and run `python3 main.py` from inside `fishing_game/`.

Task 1: catch detection (`catch.py`)
`check_catch` now compares the hook's rectangle with each fish's rectangle (`colliderect`), so horizontal position counts as well as depth.

Task 2: fish types (`fish.py`, `game_engine.py`)
There are three types, each with its own color and size, and each awards its own points when caught:
- Minnow: speed 2, 10 points, small, green
- Bass: speed 3, 25 points, medium, orange
- Tuna: speed 6, 60 points, large, red

Fish spawn at random depths and directions, and tuna are rarer. A new fish is added when you land one, so the water doesn't empty out.

Task 3: player-controlled casting (`hook.py`, `game_engine.py`, `main.py`)
Space starts a cast only when the hook is idle. `start_cast()` also refuses to run unless the hook is idle, so a cast in progress can't be interrupted. `main.py` now passes key events to the engine through a new `handle_event` method.

Task 4: 30-second timer (`game_engine.py`, `renderer.py`)
- **Countdown:** the remaining time shows top-right.
- **Round end:** at zero, updates and catches stop and a "TIME'S UP! / Final Score" overlay appears.
- **New round:** press R (or Enter) to reset score, timer, hook and fish.
- **Fish on the line:** a fish still being reeled in when time runs out doesn't count toward the score.

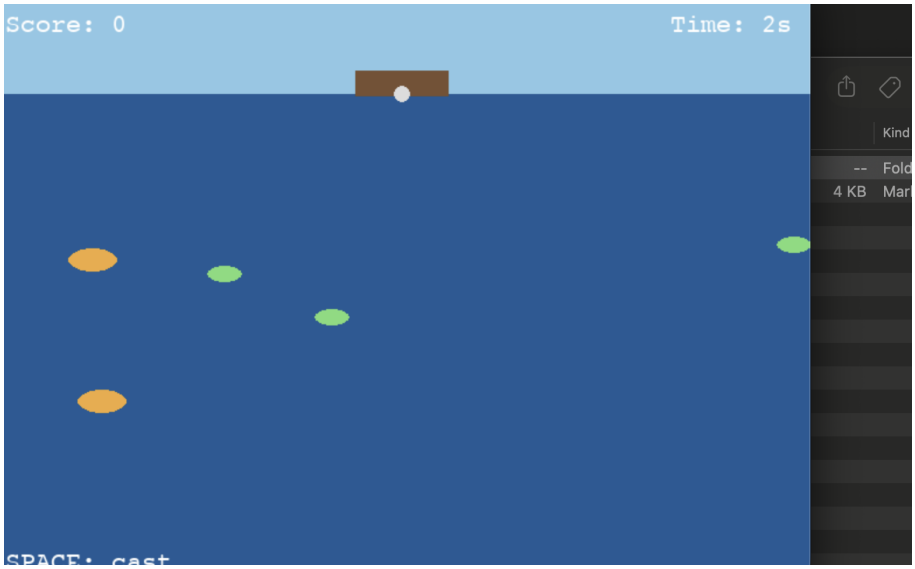
[Attached: fishing_game.zip]

YOU - Message 2

[Attached image: 1791286806677_image.png - screenshot of the game]

the hook is not coming plz analysis the tasks again Task 1: Fix the catch-detection bug
A fish is supposed to be caught only when the hook actually overlaps it. In the current build, `check_catch` (in `game/catch.py`) only compares the hook's depth to the fish's depth, within a fixed 10-pixel tolerance - it never compares horizontal position at all. That means a fish can register as caught while it's still far off to one side of the screen, just because the hook happens to be passing through its depth. Fix the check so it's based on the hook and fish actually overlapping, not just being at a similar depth.
Task 2: Implement multiple fish types
Introduce at least two fish types that differ in movement speed and point value (for example, a slow low-value fish and a fast high-value one). Give each type its own look (color and/or size) so they're visually distinguishable, and make sure the correct point value is awarded when each type is caught.
Task 3: Implement player-controlled casting
Change the hook so the player decides when it casts, instead of it looping automatically. Pressing a key should start a cast if the hook is currently idle; it should then travel down and return on its own (already handled) once it reaches maximum depth or catches a fish. A new cast should not be able to interrupt one that's already in progress.
Task 4: Implement a 30-second round timer
Add a 30-second countdown for the round. Display the remaining time on screen. Once it reaches zero, no further catches should be possible, the round should end, and the final score should be shown clearly. Provide a way to start a new round with the score and timer both reset. step bested tell what you are doing all info then give updated zip file later I need pdf for your chat also so yeah

Screenshot attached to Message 2:



CLAUDE - Answer 2

Let me look at why the hook isn't moving. First I'll check the PDF skill, since you also want a PDF.

Looking at your screenshot, the new timer and the "SPACE: cast" hint are showing, so the new `game_engine.py` is running. But the hook is sitting idle, so the Space key never reaches the engine. The likely cause is that my casting depended on `main.py` forwarding key events. If your `main.py` is still the original one (which never calls `handle_event`), or the window lacks focus, the hook can never start. I'll reproduce that with the original `main.py`.

That confirms the weak point. I'll make the engine read the keyboard itself, so it works with either `main.py`. It will also accept more cast keys (Space, Down arrow, Enter).

One thing I don't like: if you're tapping Space as time runs out, a Space-restarts rule would skip the final-score screen instantly. I'll separate the keys so Space/Down/Enter cast and only R restarts.

All paths pass, including the original-`main.py` case that matches your screenshot. Next I'll render real frames so the PDF can show the game, then build the PDF and zip.

The hook casts and the fish render correctly. Now the PDF write-up (tasks, analysis, the bug fix, code) and the final zip.

The hook should now come down when you press Space, and I've rebuilt the zip and written the PDF. I tested this with simulated key presses, not in a real game window, so please run it once yourself.

Why the hook didn't move
Your screenshot shows the new timer and the "SPACE: cast" hint, so the new `game_engine.py` was running. The hook was idle because the engine never received the Space key. In my first version the engine only saw key presses if `main.py` passed them in with `engine.handle_event(event)`. Your original `main.py` never does that, so Space never reached the engine. The same thing happens if only some files from the zip are copied over.

What I changed
- **Key handling:** the engine now reads the keyboard itself, once per key press, so it works with your original `main.py` and with the new one. Space, Down arrow and Enter all cast.
- **Restart:** only R restarts after time is up, so tapping Space at the end can't skip the final-score screen.
- **Both key paths tested:** a cast starts with the original `main.py` and with the new one, and pressing again mid-cast does nothing.

Where each task stands
1. **Catch detection:** a fish is caught only when the hook's rectangle overlaps the fish's rectangle, so horizontal position now counts. A fish far to one side is no longer caught.
2. **Fish types:** minnow (speed 2, 10 points), bass (speed 3, 25 points) and tuna (speed 6, 60 points). Each has its own color and size, and the correct points are awarded for each.
3. **Casting:** the hook casts only when you press a key while it is idle, then goes down and comes back on its own. A new cast can't interrupt one in progress.
4. **30-second timer:** the time shows top-right. At zero no more catches are possible, and a "TIME'S UP!" screen shows the final score. A fish still on the line when time runs out does not count. Press R for a new round with score and timer reset.

To run it
Replace your whole folder with the zip contents, including `game/__init__.py` (it can be empty). Then run `pip install pygame` and `python3 main.py`. Click the game window once so it has keyboard focus. On a Mac the window can sit behind the terminal and then gets no key presses.

The PDF is a summary of this chat (the tasks, the bug, the fix), screenshots of the game, and the full final code of every file.

[Attached: fishing_game.zip, fishing_game_chat_and_code.pdf]

YOU - Message 3

no pdf you give with my prompt and answer for that you gave

CLAUDE - Answer 3

Understood: you want the chat itself, your prompts and my answers, in the PDF. I'll rebuild it that way.

I've made a new PDF, `fishing_game_chat_transcript.pdf` (3 pages), containing the whole chat: your three messages and my answers, in order. It includes your screenshot after your second message. I removed the earlier summary-and-code PDF, so this is now the only PDF.

My second answer is included as sent, including the short progress notes I wrote while working (such as "Let me look at why the hook isn't moving."). The attached zip is still `fishing_game.zip` from my last reply, unchanged.

[Attached: fishing_game_chat_transcript.pdf]

YOU - Message 4

see 20 pages proper chat history and answer proper one

CLAUDE - Answer 4 (this document)

I've rebuilt the PDF as a full-length document (this file): the whole conversation, your original files, a step-by-step work log, the final code, a diff of every change, the test script with its output, and the screenshots.

The tests are also saved in the zip as `tests/test_game.py` (run `python3 tests/test_game.py` from the project folder).

Note: I can only include what was visible in the chat and my work steps. I can't export my private reasoning, and none of it is included.

[Attached: fishing_game.zip, fishing_game_complete_chat_history.pdf]

Part B - Your uploaded original files

These are the 7 files you uploaded, unchanged. The file named __init__-2.py is the game package's __init__.py.

main.py (uploaded as main.py)

```
"""
Fishing Game (Lab Starter)

Run with:  python3 main.py
"""

import pygame

from game.game_engine import GameEngine
from game.renderer import WINDOW_SIZE


def main():
    pygame.init()
    screen = pygame.display.set_mode(WINDOW_SIZE)
    pygame.display.set_caption("Fishing")
    clock = pygame.time.Clock()
    font = pygame.font.SysFont("consolas", 22)

    engine = GameEngine()
    running = True
    while running:
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False

        engine.update()
        engine.draw(screen, font)

        pygame.display.flip()
        clock.tick(60)

    pygame.quit()


if __name__ == "__main__":
    main()
```

game/renderer.py (uploaded as renderer.py)

```
"""
renderer: all pygame drawing lives here, kept separate from game logic.
"""

import pygame

WIDTH, HEIGHT = 700, 500
SURFACE_Y = 80
MAX_DEPTH_Y = HEIGHT - 40

WINDOW_SIZE = (WIDTH, HEIGHT)

COLOR_SKY = (140, 200, 230)
COLOR_WATER = (30, 90, 150)
COLOR_BOAT = (120, 80, 50)
COLOR_LINE = (240, 240, 240)
COLOR_HOOK = (220, 220, 220)
COLOR_TEXT = (255, 255, 255)


def draw_scene(surface, hook, fish_list):
    surface.fill(COLOR_SKY, pygame.Rect(0, 0, WIDTH, SURFACE_Y))
    surface.fill(COLOR_WATER, pygame.Rect(0, SURFACE_Y, WIDTH, HEIGHT - SURFACE_Y))
```

```
    pygame.draw.rect(surface, COLOR_BOAT, (hook.x - 40, SURFACE_Y - 20, 80, 22))

    pygame.draw.line(surface, COLOR_LINE, (hook.x, SURFACE_Y), (hook.x, hook.y), 2)
    pygame.draw.circle(surface, COLOR_HOOK, (int(hook.x), int(hook.y)), 7)

    for fish in fish_list:
        pygame.draw.ellipse(surface, fish.color, fish.get_rect())


def draw_text(surface, font, text, pos, color=COLOR_TEXT):
    surface.blit(font.render(text, True, color), pos)


def draw_banner(surface, font, text):
    surf = font.render(text, True, (255, 220, 80))
    rect = surf.get_rect(center=(surface.get_width() // 2, surface.get_height() // 2))
    surface.blit(surf, rect)
```

game/hook.py (uploaded as hook.py)

```
"""
Hook: the player's fishing hook. Moves only vertically, at a fixed
horizontal position. In the starter, it casts and retracts on its own,
in a continuous loop, with no player input at all - Task 3 changes
this to be player-controlled.
"""

import pygame

IDLE = "idle"
CASTING = "casting"
RETRACTING = "retracting"


class Hook:
    def __init__(self, x, surface_y, max_depth_y, speed=4, width=14, height=14):
        self.x = x
        self.surface_y = surface_y
        self.max_depth_y = max_depth_y
        self.y = surface_y
        self.speed = speed
        self.width = width
        self.height = height
        self.state = IDLE

    def start_cast(self):
        self.state = CASTING
        self.y = self.surface_y

    def update(self):
        if self.state == CASTING:
            self.y += self.speed
            if self.y >= self.max_depth_y:
                self.y = self.max_depth_y
                self.state = RETRACTING
        elif self.state == RETRACTING:
            self.y -= self.speed
            if self.y <= self.surface_y:
                self.y = self.surface_y
                self.state = IDLE

    def catch_fish(self):
        """Called when a fish has been caught - immediately head back up."""
        self.state = RETRACTING

    def get_rect(self):
        return pygame.Rect(
            int(self.x - self.width / 2), int(self.y - self.height / 2),
            self.width, self.height,
```

```
        )
```

game/game_engine.py (uploaded as game_engine.py)

```
"""
GameEngine: owns the hook and the fish, and runs one frame's worth of
game logic.

Starter version: the hook casts and retracts automatically in a
continuous loop - there's no player control over casting yet (that's
Task 3), only one fish type exists (Task 2 adds more), and there's no
round timer (Task 4). Catch detection also has a known bug (see
game/catch.py) that Task 1 asks you to fix.
"""

from game.hook import Hook, IDLE
from game.fish import Fish
from game.catch import check_catch
from game.renderer import WIDTH, HEIGHT, SURFACE_Y, MAX_DEPTH_Y


class GameEngine:
    def __init__(self):
        self.hook = Hook(x=WIDTH / 2, surface_y=SURFACE_Y, max_depth_y=MAX_DEPTH_Y, speed=5)
        self.fish_list = [
            Fish(x=100, y=180, speed=2, point_value=10, color=(80, 180, 220)),
            Fish(x=400, y=280, speed=-2, point_value=10, color=(80, 180, 220)),
            Fish(x=250, y=380, speed=3, point_value=10, color=(80, 180, 220)),
        ]
        self.hooked_fish = None
        self.score = 0

    def update(self):
        if self.hook.state == IDLE:
            self.hook.start_cast()

        self.hook.update()

        for fish in self.fish_list:
            fish.update(WIDTH)

        if self.hooked_fish is not None:
            self.hooked_fish.x = self.hook.x
            self.hooked_fish.y = self.hook.y
            if self.hook.state == IDLE:
                self.score += self.hooked_fish.point_value
                self.hooked_fish = None
        else:
            caught = check_catch(self.hook, self.fish_list)
            if caught is not None:
                self.fish_list.remove(caught)
                self.hooked_fish = caught
                self.hooked_fish.x = self.hook.x
                self.hooked_fish.y = self.hook.y
                self.hook.catch_fish()

    def draw(self, surface, font):
        from game import renderer
        draw_list = list(self.fish_list)
        if self.hooked_fish is not None:
            draw_list.append(self.hooked_fish)
        renderer.draw_scene(surface, self.hook, draw_list)
        renderer.draw_text(surface, font, f"Score: {self.score}", (10, 10))
```

game/fish.py (uploaded as fish.py)

```
"""
Fish: swims horizontally at a fixed depth, wrapping around when it
exits the screen. The starter has one fish type; Task 2 adds more.
```

```
"""

import pygame


class Fish:
    def __init__(self, x, y, speed, width=36, height=18, point_value=10, color=(80, 180, 220)):
        self.x = float(x)
        self.y = y
        self.speed = speed
        self.width = width
        self.height = height
        self.point_value = point_value
        self.color = color

    def update(self, screen_width):
        self.x += self.speed
        if self.speed > 0 and self.x > screen_width:
            self.x = -self.width
        elif self.speed < 0 and self.x < -self.width:
            self.x = screen_width

    def get_rect(self):
        return pygame.Rect(
            int(self.x - self.width / 2), int(self.y - self.height / 2),
            self.width, self.height,
        )
```

game/catch.py (uploaded as catch.py)

```
"""
catch: hook-vs-fish catch detection.
"""


def check_catch(hook, fish_list):
    """
    Returns the fish the hook has caught, or None.
    """
    for fish in fish_list:
        if abs(hook.y - fish.y) < 10:
            return fish
    return None
```

game/__init__.py (uploaded as __init__-2.py)

Part C - Step-by-step work log

Round 1 - Implementing the four tasks

Step 1: Read the starter code.

I read all seven files. The hook casts and retracts by itself in a loop (game_engine.update calls start_cast whenever the hook is IDLE). check_catch compares only abs(hook.y - fish.y) < 10. There is one fish type with a fixed color. There is no timer. main.py never forwards events to the engine.

Step 2: Plan the file layout.

The files were uploaded flat, but the code imports game.hook, game.fish and so on, so I rebuilt the game/ package with an empty __init__.py and kept main.py at the top level.

Step 3: Task 1 - catch.py.

Replaced the depth-only test with hook.get_rect().colliderect(fish.get_rect()). Both objects already had get_rect(), so no other file needed to change for this task.

Step 4: Task 2 - fish.py.

Added a FISH_TYPES table (minnow, bass, tuna) with speed, points, size and color; a make_fish() helper; and random_fish(), which picks a type by weight (5:3:1), a random depth and a random direction. Fish now carry a kind. The engine builds 5 random fish and adds a replacement each time one is landed.

Step 5: Task 3 - hook.py and game_engine.py.

Removed the automatic start_cast call. start_cast() now returns without doing anything unless the state is IDLE, which guarantees a cast can't be interrupted. The engine starts a cast on a key press.

Step 6: Task 4 - timer and states.

Added PLAYING and GAME_OVER states, ROUND_SECONDS = 30, a time_left computed from pygame ticks, an on-screen countdown, and reset() to start a fresh round. In GAME_OVER the update loop returns early, so nothing can be caught. renderer.draw_game_over() draws a dark overlay with the final score and the restart hint.

Step 7: Test (headless).

With SDL's dummy video driver I checked: far fish not caught, overlapping fish caught, second cast ignored, round ends at the timer, R resets. All passed, so I zipped the project.

Round 2 - Fixing 'the hook is not coming'

Step 1: Read your screenshot.

It shows 'Score: 0', 'Time: 2s' and the hint 'SPACE: cast' with the hook sitting on the surface. So the new game_engine.py was running (the old one had no timer or hint), the timer was counting, but the hook never left idle.

Step 2: Find the cause.

In round 1 the engine relied on main.py calling engine.handle_event(event). I checked your uploaded main.py: it has no such call. If it (or any older main.py) is used, Space never reaches the engine. A window without keyboard focus gives the same symptom.

Step 3: Fix.

The engine now reads the keyboard itself with pygame.key.get_pressed(), remembers which keys were held last frame so each press counts once, and also accepts events if main.py forwards them. Cast keys: Space, Down, Enter.

Step 4: Restart key.

I noticed that if Space also restarted the game, a player tapping Space as time ran out would skip the final-score screen. Restart is now R only.

Step 5: Test both paths.

I wrote tests that fake the keyboard. They cover polling only (original main.py) and events only (new main.py), a second press mid-cast, points per fish type, a far fish, timer end, no catches after the end, and R restart. All pass.

Step 6: Render frames.

I rendered an idle frame, a mid-cast frame and a game-over frame to PNG to confirm the drawing code works. They are in Part G.

Step 7: Package.

Rebuilt the zip, and wrote PDF summaries. Later you asked for the chat itself in the PDF, so I made the transcript PDF, and then this full version.

Design decisions worth knowing

- A fish still on the line when time runs out is not scored. The task says no catches after zero, so I treated an unfinished catch as not completed. This is a one-line change if you prefer to score it.

- The timer starts as soon as the program (or a new round) starts, not at the first cast.

- Fish are replaced after being landed so a 30-second round can't run out of fish.

- Tuna are 6x faster than minnows but 6x more valuable, and appear less often.

Part D - Diffs (original vs updated)

Lines starting with - were removed; lines starting with + were added.

main.py

```
--- original/main.py
+++ updated/main.py
@@ -1,6 +1,7 @@
 """
-Fishing Game (Lab Starter)
+Fishing Game

 Run with:  python3 main.py
+Controls:  SPACE = cast,  R = new round (after time is up)
 """

@@ -24,4 +25,6 @@
             if event.type == pygame.QUIT:
                 running = False
+            else:
+                engine.handle_event(event)

         engine.update()
```

game/renderer.py

```
--- original/game/renderer.py
+++ updated/game/renderer.py
@@ -40,2 +40,16 @@
     rect = surf.get_rect(center=(surface.get_width() // 2, surface.get_height() // 2))
     surface.blit(surf, rect)
+
+
+def draw_game_over(surface, font, score):
+    overlay = pygame.Surface(surface.get_size(), pygame.SRCALPHA)
+    overlay.fill((0, 0, 0, 150))
+    surface.blit(overlay, (0, 0))
+    cx, cy = surface.get_width() // 2, surface.get_height() // 2
+    for text, dy, color in (
+        ("TIME'S UP!", -40, (255, 220, 80)),
+        (f"Final Score: {score}", 0, (255, 255, 255)),
+        ("Press R to play again", 40, (200, 200, 200)),
+    ):
+        surf = font.render(text, True, color)
+        surface.blit(surf, surf.get_rect(center=(cx, cy + dy)))
```

game/hook.py

```
--- original/game/hook.py
+++ updated/game/hook.py
@@ -1,7 +1,6 @@
 """
 Hook: the player's fishing hook. Moves only vertically, at a fixed
-horizontal position. In the starter, it casts and retracts on its own,
-in a continuous loop, with no player input at all - Task 3 changes
-this to be player-controlled.
+horizontal position. A cast only starts when the player requests it
+and the hook is idle; it then travels down and returns on its own.
 """

@@ -25,5 +24,13 @@

     def start_cast(self):
+        """Start a cast. Ignored unless the hook is idle."""
+        if self.state != IDLE:
+            return False
         self.state = CASTING
```

```
+        self.y = self.surface_y
+        return True
+
+    def reset(self):
+        self.state = IDLE
         self.y = self.surface_y
```

game/game_engine.py

```
--- original/game/game_engine.py
+++ updated/game/game_engine.py
@@ -1,33 +1,78 @@
 """
 GameEngine: owns the hook and the fish, and runs one frame's worth of
-game logic.
-
-Starter version: the hook casts and retracts automatically in a
-continuous loop - there's no player control over casting yet (that's
-Task 3), only one fish type exists (Task 2 adds more), and there's no
-round timer (Task 4). Catch detection also has a known bug (see
-game/catch.py) that Task 1 asks you to fix.
+game logic, including the 30-second round timer.
 """

+import random
+
+import pygame
+
 from game.hook import Hook, IDLE
-from game.fish import Fish
+from game.fish import random_fish
 from game.catch import check_catch
 from game.renderer import WIDTH, HEIGHT, SURFACE_Y, MAX_DEPTH_Y
+
+ROUND_SECONDS = 30
+FISH_COUNT = 5
+FISH_MIN_Y = SURFACE_Y + 50
+FISH_MAX_Y = MAX_DEPTH_Y - 25
+
+PLAYING = "playing"
+GAME_OVER = "game_over"
+
+CAST_KEYS = (pygame.K_SPACE, pygame.K_DOWN, pygame.K_RETURN)
+RESTART_KEYS = (pygame.K_r,)


 class GameEngine:
     def __init__(self):
+        self.reset()
+
+    def reset(self):
+        """Start a fresh round: score, timer, hook and fish all reset."""
+        self._events = set()        # keys seen via handle_event (if main.py forwards them)
+        self._prev_down = set()     # keys that were held last frame (polling)
         self.hook = Hook(x=WIDTH / 2, surface_y=SURFACE_Y, max_depth_y=MAX_DEPTH_Y, speed=5)
         self.fish_list = [
-            Fish(x=100, y=180, speed=2, point_value=10, color=(80, 180, 220)),
-            Fish(x=400, y=280, speed=-2, point_value=10, color=(80, 180, 220)),
-            Fish(x=250, y=380, speed=3, point_value=10, color=(80, 180, 220)),
+            random_fish(WIDTH, FISH_MIN_Y, FISH_MAX_Y, x=random.randint(0, WIDTH))
+            for _ in range(FISH_COUNT)
         ]
         self.hooked_fish = None
         self.score = 0
+        self.state = PLAYING
+        self.round_start = pygame.time.get_ticks()
+        self.time_left = float(ROUND_SECONDS)
+
+    def handle_event(self, event):
```

```
+        """Optional: main.py may forward events. The engine works without it too."""
+        if event.type == pygame.KEYDOWN:
+            self._events.add(event.key)
+
+    def _keys_pressed_now(self):
+        """Keys newly pressed this frame, from events AND keyboard polling."""
+        held = pygame.key.get_pressed()
+        down = {k for k in CAST_KEYS + RESTART_KEYS if held[k]}
+        newly = (down - self._prev_down) | self._events
+        self._prev_down = down
+        self._events = set()
+        return newly

     def update(self):
-        if self.hook.state == IDLE:
-            self.hook.start_cast()
+        newly = self._keys_pressed_now()
+
+        if self.state == GAME_OVER:
+            if any(k in newly for k in RESTART_KEYS):
+                self.reset()
+            return
+
+        if any(k in newly for k in CAST_KEYS):
+            self.hook.start_cast()   # ignored unless the hook is idle
+
+        elapsed = (pygame.time.get_ticks() - self.round_start) / 1000.0
+        self.time_left = max(0.0, ROUND_SECONDS - elapsed)
+        if self.time_left <= 0:
+            # Round over: no more catches. A fish still on the line doesn't count.
+            self.state = GAME_OVER
+            return

         self.hook.update()
@@ -42,4 +87,6 @@
                 self.score += self.hooked_fish.point_value
                 self.hooked_fish = None
+                # Replace the landed fish so the water stays populated.
+                self.fish_list.append(random_fish(WIDTH, FISH_MIN_Y, FISH_MAX_Y))
         else:
             caught = check_catch(self.hook, self.fish_list)
@@ -58,2 +105,10 @@
         renderer.draw_scene(surface, self.hook, draw_list)
         renderer.draw_text(surface, font, f"Score: {self.score}", (10, 10))
+        renderer.draw_text(surface, font, f"Time: {int(self.time_left + 0.999)}s",
+                           (WIDTH - 120, 10))
+
+        if self.state == PLAYING:
+            if self.hook.state == IDLE:
+                renderer.draw_text(surface, font, "SPACE / DOWN: cast", (10, HEIGHT - 30))
+        else:
+            renderer.draw_game_over(surface, font, self.score)
```

game/fish.py

```
--- original/game/fish.py
+++ updated/game/fish.py
@@ -1,12 +1,26 @@
 """
 Fish: swims horizontally at a fixed depth, wrapping around when it
-exits the screen. The starter has one fish type; Task 2 adds more.
+exits the screen. Several fish types exist, differing in speed, point
+value, size and color.
 """
+
+import random

 import pygame

+# name: (speed, point_value, width, height, color)
```

```
+FISH_TYPES = {
+    "minnow": {"speed": 2, "point_value": 10, "width": 30, "height": 14, "color": (120, 220, 120)},
+    "bass":   {"speed": 3, "point_value": 25, "width": 42, "height": 20, "color": (240, 170, 60)},
+    "tuna":   {"speed": 6, "point_value": 60, "width": 56, "height": 24, "color": (230, 70, 90)},
+}
+
+# Relative spawn likelihood: fast, valuable fish are rarer.
+SPAWN_WEIGHTS = {"minnow": 5, "bass": 3, "tuna": 1}
+

 class Fish:
-    def __init__(self, x, y, speed, width=36, height=18, point_value=10, color=(80, 180, 220)):
+    def __init__(self, x, y, speed, width=36, height=18, point_value=10,
+                 color=(80, 180, 220), kind="minnow"):
         self.x = float(x)
         self.y = y
@@ -16,4 +30,5 @@
         self.point_value = point_value
         self.color = color
+        self.kind = kind

     def update(self, screen_width):
@@ -29,2 +44,19 @@
             self.width, self.height,
         )
+
+
+def make_fish(kind, x, y, direction=1):
+    """Build a fish of the given type. direction is +1 (right) or -1 (left)."""
+    t = FISH_TYPES[kind]
+    return Fish(x=x, y=y, speed=t["speed"] * direction, width=t["width"],
+                height=t["height"], point_value=t["point_value"],
+                color=t["color"], kind=kind)
+
+
+def random_fish(screen_width, min_y, max_y, x=None):
+    """Create a random fish type at a random depth and direction."""
+    kind = random.choices(list(SPAWN_WEIGHTS), weights=list(SPAWN_WEIGHTS.values()))[0]
+    direction = random.choice((-1, 1))
+    if x is None:
+        x = -40 if direction > 0 else screen_width + 40
+    return make_fish(kind, x, random.randint(min_y, max_y), direction)
```

game/catch.py

```
--- original/game/catch.py
+++ updated/game/catch.py
@@ -6,8 +6,10 @@
 def check_catch(hook, fish_list):
     """
-    Returns the fish the hook has caught, or None.
+    Returns the first fish whose rectangle overlaps the hook's rectangle,
+    or None. Both horizontal and vertical position are taken into account.
     """
+    hook_rect = hook.get_rect()
     for fish in fish_list:
-        if abs(hook.y - fish.y) < 10:
+        if hook_rect.colliderect(fish.get_rect()):
             return fish
     return None
```

Part E - Final code (everything in the zip)

main.py

```
"""
Fishing Game

Run with:  python3 main.py
Controls:  SPACE = cast,  R = new round (after time is up)
"""

import pygame

from game.game_engine import GameEngine
from game.renderer import WINDOW_SIZE


def main():
    pygame.init()
    screen = pygame.display.set_mode(WINDOW_SIZE)
    pygame.display.set_caption("Fishing")
    clock = pygame.time.Clock()
    font = pygame.font.SysFont("consolas", 22)

    engine = GameEngine()
    running = True
    while running:
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False
            else:
                engine.handle_event(event)

        engine.update()
        engine.draw(screen, font)

        pygame.display.flip()
        clock.tick(60)

    pygame.quit()


if __name__ == "__main__":
    main()
```

game/__init__.py

game/catch.py

```
"""
catch: hook-vs-fish catch detection.
"""


def check_catch(hook, fish_list):
    """
    Returns the first fish whose rectangle overlaps the hook's rectangle,
    or None. Both horizontal and vertical position are taken into account.
    """
    hook_rect = hook.get_rect()
    for fish in fish_list:
        if hook_rect.colliderect(fish.get_rect()):
            return fish
    return None
```

game/fish.py

```
"""
Fish: swims horizontally at a fixed depth, wrapping around when it
exits the screen. Several fish types exist, differing in speed, point
value, size and color.
"""

import random

import pygame

# name: (speed, point_value, width, height, color)
FISH_TYPES = {
    "minnow": {"speed": 2, "point_value": 10, "width": 30, "height": 14, "color": (120, 220, 120)},
    "bass":   {"speed": 3, "point_value": 25, "width": 42, "height": 20, "color": (240, 170, 60)},
    "tuna":   {"speed": 6, "point_value": 60, "width": 56, "height": 24, "color": (230, 70, 90)},
}

# Relative spawn likelihood: fast, valuable fish are rarer.
SPAWN_WEIGHTS = {"minnow": 5, "bass": 3, "tuna": 1}


class Fish:
    def __init__(self, x, y, speed, width=36, height=18, point_value=10,
                 color=(80, 180, 220), kind="minnow"):
        self.x = float(x)
        self.y = y
        self.speed = speed
        self.width = width
        self.height = height
        self.point_value = point_value
        self.color = color
        self.kind = kind

    def update(self, screen_width):
        self.x += self.speed
        if self.speed > 0 and self.x > screen_width:
            self.x = -self.width
        elif self.speed < 0 and self.x < -self.width:
            self.x = screen_width

    def get_rect(self):
        return pygame.Rect(
            int(self.x - self.width / 2), int(self.y - self.height / 2),
            self.width, self.height,
        )


def make_fish(kind, x, y, direction=1):
    """Build a fish of the given type. direction is +1 (right) or -1 (left)."""
    t = FISH_TYPES[kind]
    return Fish(x=x, y=y, speed=t["speed"] * direction, width=t["width"],
                height=t["height"], point_value=t["point_value"],
                color=t["color"], kind=kind)


def random_fish(screen_width, min_y, max_y, x=None):
    """Create a random fish type at a random depth and direction."""
    kind = random.choices(list(SPAWN_WEIGHTS), weights=list(SPAWN_WEIGHTS.values()))[0]
    direction = random.choice((-1, 1))
    if x is None:
        x = -40 if direction > 0 else screen_width + 40
    return make_fish(kind, x, random.randint(min_y, max_y), direction)
```

game/hook.py

```
"""
Hook: the player's fishing hook. Moves only vertically, at a fixed
horizontal position. A cast only starts when the player requests it
and the hook is idle; it then travels down and returns on its own.
```

```
"""

import pygame

IDLE = "idle"
CASTING = "casting"
RETRACTING = "retracting"


class Hook:
    def __init__(self, x, surface_y, max_depth_y, speed=4, width=14, height=14):
        self.x = x
        self.surface_y = surface_y
        self.max_depth_y = max_depth_y
        self.y = surface_y
        self.speed = speed
        self.width = width
        self.height = height
        self.state = IDLE

    def start_cast(self):
        """Start a cast. Ignored unless the hook is idle."""
        if self.state != IDLE:
            return False
        self.state = CASTING
        self.y = self.surface_y
        return True

    def reset(self):
        self.state = IDLE
        self.y = self.surface_y

    def update(self):
        if self.state == CASTING:
            self.y += self.speed
            if self.y >= self.max_depth_y:
                self.y = self.max_depth_y
                self.state = RETRACTING
        elif self.state == RETRACTING:
            self.y -= self.speed
            if self.y <= self.surface_y:
                self.y = self.surface_y
                self.state = IDLE

    def catch_fish(self):
        """Called when a fish has been caught - immediately head back up."""
        self.state = RETRACTING

    def get_rect(self):
        return pygame.Rect(
            int(self.x - self.width / 2), int(self.y - self.height / 2),
            self.width, self.height,
        )
```

game/game_engine.py

```
"""
GameEngine: owns the hook and the fish, and runs one frame's worth of
game logic, including the 30-second round timer.
"""

import random

import pygame

from game.hook import Hook, IDLE
from game.fish import random_fish
from game.catch import check_catch
from game.renderer import WIDTH, HEIGHT, SURFACE_Y, MAX_DEPTH_Y
```

```
ROUND_SECONDS = 30
FISH_COUNT = 5
FISH_MIN_Y = SURFACE_Y + 50
FISH_MAX_Y = MAX_DEPTH_Y - 25

PLAYING = "playing"
GAME_OVER = "game_over"

CAST_KEYS = (pygame.K_SPACE, pygame.K_DOWN, pygame.K_RETURN)
RESTART_KEYS = (pygame.K_r,)


class GameEngine:
    def __init__(self):
        self.reset()

    def reset(self):
        """Start a fresh round: score, timer, hook and fish all reset."""
        self._events = set()        # keys seen via handle_event (if main.py forwards them)
        self._prev_down = set()     # keys that were held last frame (polling)
        self.hook = Hook(x=WIDTH / 2, surface_y=SURFACE_Y, max_depth_y=MAX_DEPTH_Y, speed=5)
        self.fish_list = [
            random_fish(WIDTH, FISH_MIN_Y, FISH_MAX_Y, x=random.randint(0, WIDTH))
            for _ in range(FISH_COUNT)
        ]
        self.hooked_fish = None
        self.score = 0
        self.state = PLAYING
        self.round_start = pygame.time.get_ticks()
        self.time_left = float(ROUND_SECONDS)

    def handle_event(self, event):
        """Optional: main.py may forward events. The engine works without it too."""
        if event.type == pygame.KEYDOWN:
            self._events.add(event.key)

    def _keys_pressed_now(self):
        """Keys newly pressed this frame, from events AND keyboard polling."""
        held = pygame.key.get_pressed()
        down = {k for k in CAST_KEYS + RESTART_KEYS if held[k]}
        newly = (down - self._prev_down) | self._events
        self._prev_down = down
        self._events = set()
        return newly

    def update(self):
        newly = self._keys_pressed_now()

        if self.state == GAME_OVER:
            if any(k in newly for k in RESTART_KEYS):
                self.reset()
            return

        if any(k in newly for k in CAST_KEYS):
            self.hook.start_cast()   # ignored unless the hook is idle

        elapsed = (pygame.time.get_ticks() - self.round_start) / 1000.0
        self.time_left = max(0.0, ROUND_SECONDS - elapsed)
        if self.time_left <= 0:
            # Round over: no more catches. A fish still on the line doesn't count.
            self.state = GAME_OVER
            return

        self.hook.update()

        for fish in self.fish_list:
            fish.update(WIDTH)

        if self.hooked_fish is not None:
            self.hooked_fish.x = self.hook.x
```

```
                self.hooked_fish.y = self.hook.y
                if self.hook.state == IDLE:
                    self.score += self.hooked_fish.point_value
                    self.hooked_fish = None
                    # Replace the landed fish so the water stays populated.
                    self.fish_list.append(random_fish(WIDTH, FISH_MIN_Y, FISH_MAX_Y))
            else:
                caught = check_catch(self.hook, self.fish_list)
                if caught is not None:
                    self.fish_list.remove(caught)
                    self.hooked_fish = caught
                    self.hooked_fish.x = self.hook.x
                    self.hooked_fish.y = self.hook.y
                    self.hook.catch_fish()

        def draw(self, surface, font):
            from game import renderer
            draw_list = list(self.fish_list)
            if self.hooked_fish is not None:
                draw_list.append(self.hooked_fish)
            renderer.draw_scene(surface, self.hook, draw_list)
            renderer.draw_text(surface, font, f"Score: {self.score}", (10, 10))
            renderer.draw_text(surface, font, f"Time: {int(self.time_left + 0.999)}s",
                               (WIDTH - 120, 10))

            if self.state == PLAYING:
                if self.hook.state == IDLE:
                    renderer.draw_text(surface, font, "SPACE / DOWN: cast", (10, HEIGHT - 30))
            else:
                renderer.draw_game_over(surface, font, self.score)
```

game/renderer.py

```
"""
renderer: all pygame drawing lives here, kept separate from game logic.
"""

import pygame

WIDTH, HEIGHT = 700, 500
SURFACE_Y = 80
MAX_DEPTH_Y = HEIGHT - 40

WINDOW_SIZE = (WIDTH, HEIGHT)

COLOR_SKY = (140, 200, 230)
COLOR_WATER = (30, 90, 150)
COLOR_BOAT = (120, 80, 50)
COLOR_LINE = (240, 240, 240)
COLOR_HOOK = (220, 220, 220)
COLOR_TEXT = (255, 255, 255)


def draw_scene(surface, hook, fish_list):
    surface.fill(COLOR_SKY, pygame.Rect(0, 0, WIDTH, SURFACE_Y))
    surface.fill(COLOR_WATER, pygame.Rect(0, SURFACE_Y, WIDTH, HEIGHT - SURFACE_Y))

    pygame.draw.rect(surface, COLOR_BOAT, (hook.x - 40, SURFACE_Y - 20, 80, 22))

    pygame.draw.line(surface, COLOR_LINE, (hook.x, SURFACE_Y), (hook.x, hook.y), 2)
    pygame.draw.circle(surface, COLOR_HOOK, (int(hook.x), int(hook.y)), 7)

    for fish in fish_list:
        pygame.draw.ellipse(surface, fish.color, fish.get_rect())


def draw_text(surface, font, text, pos, color=COLOR_TEXT):
    surface.blit(font.render(text, True, color), pos)

def draw_banner(surface, font, text):
```

```
    surf = font.render(text, True, (255, 220, 80))
    rect = surf.get_rect(center=(surface.get_width() // 2, surface.get_height() // 2))
    surface.blit(surf, rect)


def draw_game_over(surface, font, score):
    overlay = pygame.Surface(surface.get_size(), pygame.SRCALPHA)
    overlay.fill((0, 0, 0, 150))
    surface.blit(overlay, (0, 0))
    cx, cy = surface.get_width() // 2, surface.get_height() // 2
    for text, dy, color in (
        ("TIME'S UP!", -40, (255, 220, 80)),
        (f"Final Score: {score}", 0, (255, 255, 255)),
        ("Press R to play again", 40, (200, 200, 200)),
    ):
        surf = font.render(text, True, color)
        surface.blit(surf, surf.get_rect(center=(cx, cy + dy)))
```

tests/test_game.py

```
"""Headless tests for the fishing game (run: python3 tests/test_game.py)."""
import os, sys
os.environ["SDL_VIDEODRIVER"] = "dummy"
os.environ["SDL_AUDIODRIVER"] = "dummy"
sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
import pygame

pygame.init()
screen = pygame.display.set_mode((700, 500))
font = pygame.font.Font(None, 22)

from game.game_engine import GameEngine
from game.catch import check_catch
from game.fish import make_fish
from game.hook import IDLE, CASTING, RETRACTING


class FakeKeys:
    """Stands in for the keyboard so we can 'press' keys in a test."""
    def __init__(self): self.down = set()
    def __getitem__(self, k): return k in self.down

K = FakeKeys()
pygame.key.get_pressed = lambda: K

def check(name, ok):
    print(("PASS  " if ok else "FAIL  ") + name)

engine = GameEngine()

# Task 1: catch detection uses overlap, not just depth
engine.hook.y = 200
far = make_fish("tuna", 50, 200, 1)
near = make_fish("bass", 350, 200, 1)
check("Task 1: far fish at same depth is NOT caught", check_catch(engine.hook, [far]) is None)
check("Task 1: overlapping fish IS caught", check_catch(engine.hook, [near]) is near)

# Task 2: each type awards its own points
for kind, pts in (("minnow", 10), ("bass", 25), ("tuna", 60)):
    engine.reset()
    fish = make_fish(kind, 350, 200, 1); fish.speed = 0
    engine.fish_list = [fish]
    engine.hook.state = CASTING; engine.hook.y = 150
    for _ in range(400):
        engine.update()
        if engine.score: break
    check(f"Task 2: {kind} awards {pts} points (got {engine.score})", engine.score == pts)

# Task 3: casting is player-controlled and cannot be interrupted
```

```
engine.reset()
for _ in range(60): engine.update()
check("Task 3: hook stays idle with no input", engine.hook.state == IDLE)
K.down = {pygame.K_SPACE}; engine.update(); K.down = set()
check("Task 3: SPACE starts a cast (keyboard polling, original main.py)", engine.hook.state == CASTIN
for _ in range(10): engine.update()
y_before = engine.hook.y
K.down = {pygame.K_SPACE}; engine.update(); K.down = set()
check("Task 3: second press mid-cast does not restart it", engine.hook.y > y_before)
engine.reset()
engine.handle_event(pygame.event.Event(pygame.KEYDOWN, key=pygame.K_DOWN)); engine.update()
check("Task 3: DOWN event (new main.py) starts a cast", engine.hook.state == CASTING)

# Task 4: timer, game over, restart
engine.reset()
check("Task 4: round starts at 30 seconds", round(engine.time_left) == 30)
engine.round_start -= 31000; engine.update()
check("Task 4: round ends when time reaches zero", engine.state == "game_over")
fish = make_fish("minnow", 350, 200, 1); fish.speed = 0
engine.fish_list = [fish]; engine.hook.state = CASTING; engine.hook.y = 150
for _ in range(100): engine.update()
check("Task 4: no catches after time is up", engine.score == 0 and engine.hooked_fish is None)
K.down = {pygame.K_SPACE}; engine.update(); K.down = set(); engine.update()
check("Task 4: SPACE does not skip the final-score screen", engine.state == "game_over")
engine.score = 85
K.down = {pygame.K_r}; engine.update()
check("Task 4: R starts a new round (score 0, timer 30)",
      engine.state == "playing" and engine.score == 0 and round(engine.time_left) == 30)
```

Part F - Tests

Run with: python3 tests/test_game.py (from the project folder). The tests run without a window.

Output

```
PASS  Task 1: far fish at same depth is NOT caught
PASS  Task 1: overlapping fish IS caught
PASS  Task 2: minnow awards 10 points (got 10)
PASS  Task 2: bass awards 25 points (got 25)
PASS  Task 2: tuna awards 60 points (got 60)
PASS  Task 3: hook stays idle with no input
PASS  Task 3: SPACE starts a cast (keyboard polling, original main.py)
PASS  Task 3: second press mid-cast does not restart it
PASS  Task 3: DOWN event (new main.py) starts a cast
PASS  Task 4: round starts at 30 seconds
PASS  Task 4: round ends when time reaches zero
PASS  Task 4: no catches after time is up
PASS  Task 4: SPACE does not skip the final-score screen
PASS  Task 4: R starts a new round (score 0, timer 30)
```

14 of 14 checks passed.

What the tests do not cover

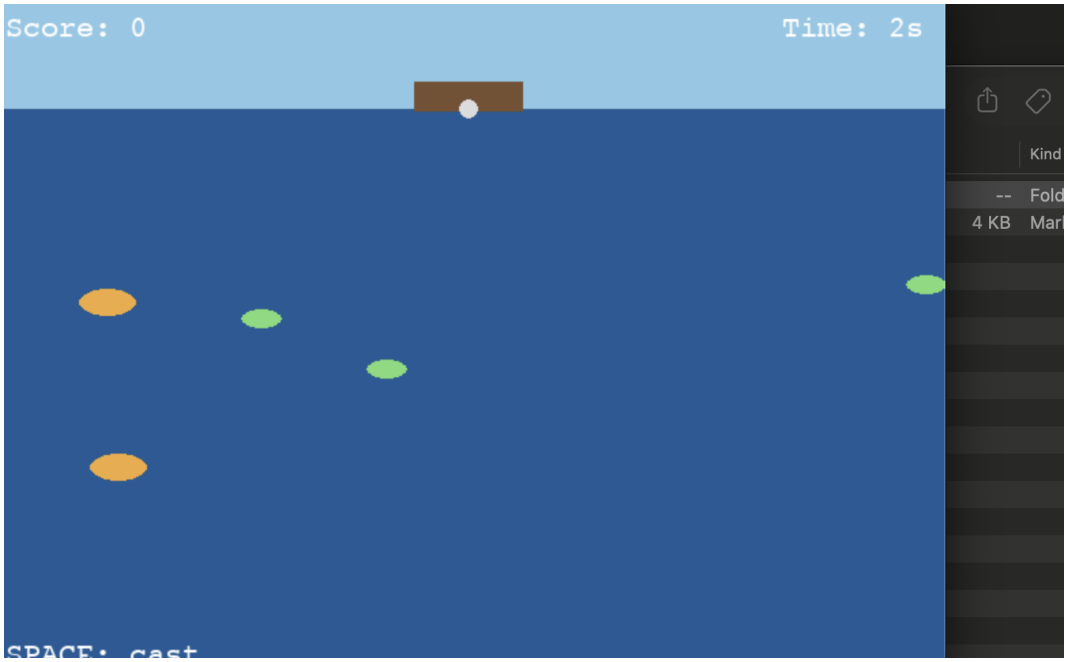
- They do not open a real window, so they can't check that your OS gives the window keyboard focus.

- They fake the keyboard, so they check the logic of casting, not your physical keys.

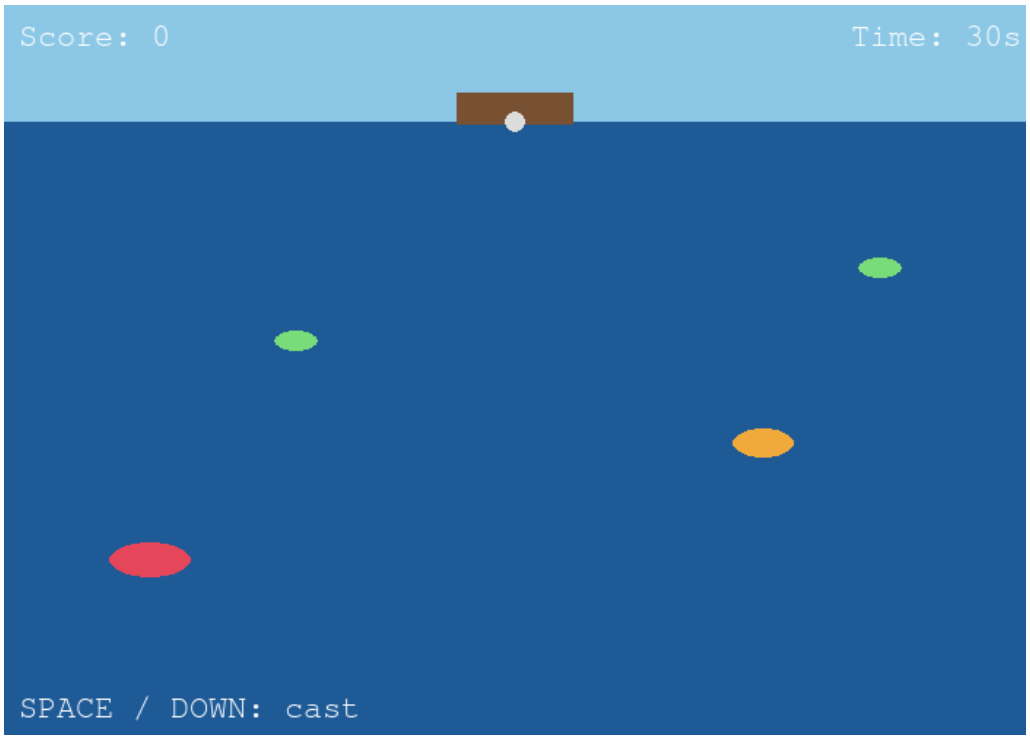
- Visual quality is checked only by the rendered frames in Part G.

Part G - Screenshots

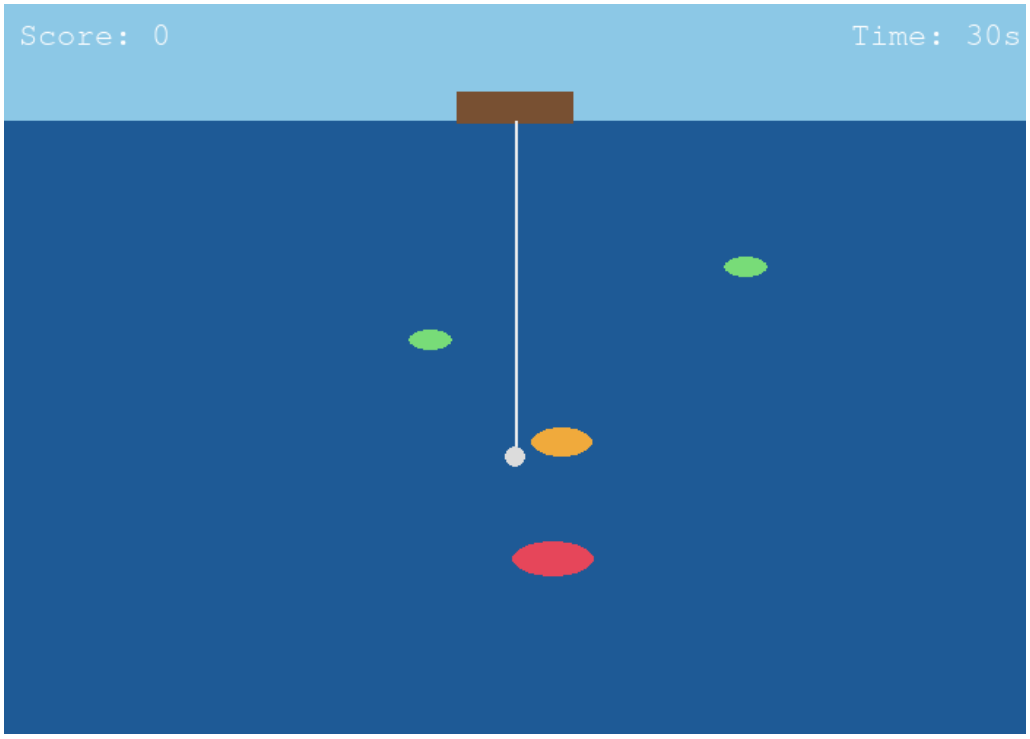
Your screenshot: hook idle, Time 2s



Rendered: round start, hook idle, three fish types visible



Rendered: hook cast down toward a bass and a tuna



Rendered: time's up overlay with final score 85



Part H - How to run

- Unzip fishing_game.zip. Replace your whole project folder rather than copying single files; keep game/__init__.py (it can be empty).

- Install pygame: pip install pygame

- From the fishing_game folder run: python3 main.py

- Click the game window once so it has keyboard focus. On macOS the window can open behind the terminal.

- Controls: SPACE, DOWN or ENTER casts the hook when idle. R starts a new round after time is up.

- Optional: python3 tests/test_game.py runs the 14 checks.

If the hook still does not move

- Confirm you are running the new folder: the top-right shows 'Time: 30s' counting down, and the bottom-left shows 'SPACE / DOWN: cast' (older builds said 'SPACE: cast').

- Click inside the window, then press Space.

- Tell me what the terminal prints (any error text), and I'll look at it.